Agentic AI

An Introduction

Filippo Lentoni

Senior Applied Scientist, Amazon
Columbia University

Agenda

1. Introduction
2. Anatomy of an Agent
3. Agent Patterns
4. When to Use Agentic AI
5. End-to-End Example
6. Notebook

Why It Matters

"OpenClaw is the most important software release probably ever"

— Jensen Huang

Why Agentic AI?

LLMs have fundamental limits:

- Knowledge frozen at pretraining cutoff
- Cannot act on the world — no file access, no APIs
- No persistent memory across sessions
- Single-pass generation

Definition

An **agent** is a software program capable of interacting with its environment and performing tasks to meet a goal. Agents have the ability to:

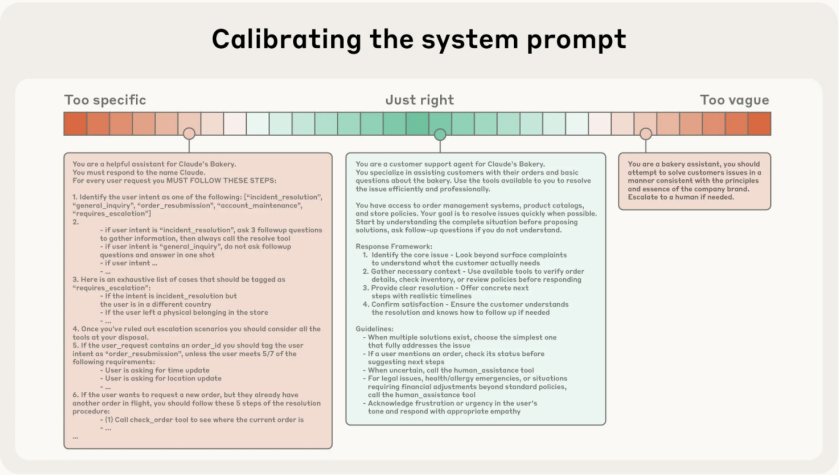
1. Understand a user request in natural language
2. Generate a plan
3. Identify the required resources (APIs, data, tools, . . .)
4. Execute the plan
5. Overcome obstacles by retrying the process

Anatomy of an Agent

- **Cognitive Layer**
- **Action Layer**
- **Memory Layer**
- **Safety Layer**

Cognitive Layer

- **Foundation model**: reasoning, planning, tool selection
- **System prompt**: prompt engineering and context engineering



At one end of the spectrum, we see brittle if-else hardcoded prompts, and at the other end we see prompts that are overly general or falsely assume shared context.

Source: Anthropic, *Effective context engineering for AI agents*
`https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents`

Action Layer

Action Layer

- **Tools**: callable capabilities that let the agent interact with the world. MCP is the open standard that lets AI applications connect in a consistent way to external tools.
- **Skills**: reusable workflows that combine reasoning and often multiple tools

Tool examples

- `search_web(query)`: retrieves recent information from the web
- `execute_sql(query)`: queries a database and returns structured results

Skill examples

- **Customer support triage**: read the ticket, classify intent, retrieve the order status, draft a response, and escalate if needed
- **Weekly business review generator**: pull metrics from dashboards, summarize deltas, identify anomalies, and draft the report

A tool is a single callable capability; a skill is a reusable workflow that combines reasoning and often multiple tools.

Tool Example - RAG: Retrieval-Augmented Generation

Goal: ground model outputs in external knowledge beyond the training cutoff

Phase 1 — Offline: Knowledge Base Preparation

1. Ingest documents
2. Chunk text *(strategy, size, overlap)*
3. Compute embeddings *(model choice)*
4. Store in vector index *(distance metric)*

Phase 2 — Online: Retrieval & Generation

1. Embed user query
2. Retrieve top-k chunks
3. Inject context into prompt
4. Generate grounded answer

Tool Example - RAG: Parameters

Phase	Parameter	What it controls	Values
Offline	Chunking strategy	How text is split (fixed, semantic, hierarchical)	Fixed-size
	Chunk size	Amount of context per chunk	512 tokens
	Chunk overlap	Context preserved across boundaries	20%
	Embedding model	How text is converted to vectors	Titan Embed v2
Online	Number of results	Chunks returned per search (top-k)	5
	Distance metric	How vectors are compared	Cosine
	Query formulation	How search queries are constructed	Model-driven
Generation	System prompt	Reasoning instructions, output format	Custom
	Model	LLM that generates the answer	Claude Sonnet 4.5

Memory Layer

Short-Term Memory Short-term memory captures turn-by-turn interactions within a single session. This lets agents maintain immediate context without requiring users to repeat information.

Example: If a user asks, "What's the weather like in Seattle?" and then follows up with "What about tomorrow?", the agent uses the recent conversation history to understand that "tomorrow" still refers to Seattle.

Long-Term Memory

Long-term memory automatically extracts and stores key insights across multiple sessions, including user preferences, important facts, and session summaries.

Example: If a customer mentions they prefer window seats during flight booking, the agent stores that preference. In future interactions, it can proactively offer window seats and create a more personalized experience.

Why Memory Matters

Without memory, every conversation starts from zero - customers repeat themselves, context is lost, and personalization is impossible. Memory enables agents to **recognize returning users**, **recall preferences automatically**, and **connect related conversations** over time.

Safety Layer

Guardrails operate on agent inputs and outputs and provide configurable safeguards to detect and mitigate risks such as:

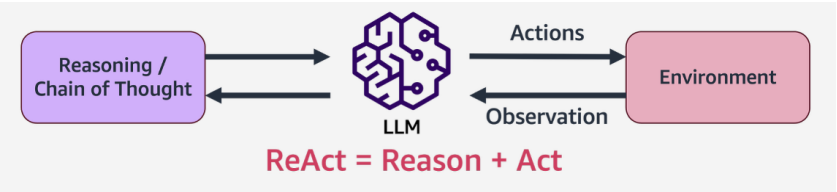
- **Prompt Injection**: malicious instructions that try to override system prompts or intended behavior
- **Jailbreaks**: adversarial prompts that attempt to bypass safety safeguards
- **PII Exposure**: sensitive personal data appearing in inputs, memory, or outputs
- **Toxicity**: harmful, abusive, or hateful language in prompts or responses
- **Prompt Leakage**: disclosure of hidden prompts, internal instructions, or private context

To further improve the safety of AI applications, developers can also control tool authorization and permissions through custom policies at the user level, ensuring that agents can access only the tools and actions allowed for a given user or role.

ReAct Framework

ReAct (Reasoning + Acting)

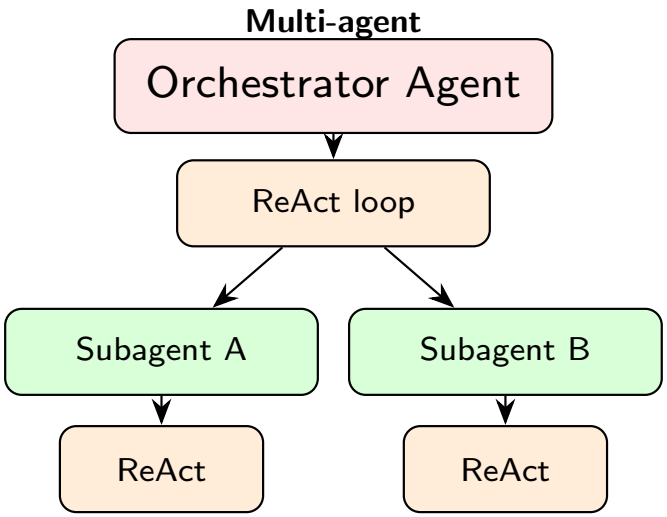
- Alternates between:
- **Reasoning** — reason about the next step
- **Act** — take an action
- **Observe** — receive result, update beliefs with real-world feedback
- **Repeat** until the goal is reached



Orchestration: Single-Agent vs. Multi-Agent

Multi-Agent System

- **Orchestrator agent**: decomposes the goal, delegates to subagents, synthesizes results
- **Subagents**: specialized, each running its own internal loop
- Enables parallelism and role separation, cost optimization
- Overhead: coordination, communication, complexity



When to Use Agents: Strategy Creation - Chatbot-Like Experience

Agents are valuable when the path to a solution is uncertain and the required knowledge or capabilities are distributed across tools, documents, databases, and other data types. In these settings, the agent can search across sources, use external capabilities, logically connect relevant information using FM reasoning, and iteratively build or refine a plan to address the user's goal. An agentic chatbot allows users to go beyond their own expertise. For example, it can help by translating content across languages or formats, exploring unfamiliar domains, surfacing relevant context, and solving problems outside the user's immediate know-how.

Examples of applications:

1. Enterprise Knowledge Assistant
2. Customer Support copilots
3. Developer Documentation Agents
4. Data Analyst Agents

When to Use Agentic AI - Decision Making: Strategy Execution

After planning, execution can follow different paths:

Case 1 — Deterministic Execution

- Plan is well-specified
- Environment is stable
- Limited unforeseen events
- Automation through scripts, workflows, decision trees
- Use the coding agent to automate the workflow

Case 2 — Runtime Agentic Execution

- Plan remains high-level
- Environment is dynamic / non-stationary
- System must adapt step by step
- Use a long-running autonomous agent (can use the coding agent as a tool)

Coding Agent & Long-Horizon Autonomous Agent

Codex (OpenAI), Claude Code (Anthropic)

January 23, 2026 Engineering

Unrolling the Codex agent loop

By Michael Bolin, Member of the Technical Staff

https://openai.com/index/unrolling-the-codex-agent-loop/

OpenClaw

OpenClaw — Personal AI Assistant

OPENCLAW

EXFOLIATE! EXFOLIATE!

BUILD FAILING | RELEASE V2026.3.13-1 | DISCORD 28K ONLINE | LICENSE MIT

OpenClaw is a *personal AI assistant* you run on your own devices. It answers you on the channels you already use (WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage, BlueBubbles, IRC, Microsoft Teams, Matrix, Feishu, LINE, Mattermost, Nextcloud Talk, Nostr, Synology Chat, Tlon, Twitch, Zalo, Zalo Personal, WebChat). It can speak and listen on macOS/iOS/Android, and can render a live Canvas you control. The Gateway is just the control plane — the product is the assistant.

https://openclaw.ai

Modern Supply Chains: Optimization, ML, and Data Complexity

Modern supply chains rely on both **mathematical optimization** and **machine learning** systems. Examples include:

- **Truck planning / allocation models** that optimize transportation resources under cost, capacity, and service constraints
- **ML classifiers** that determine whether products are eligible for air or ground transportation based on chemical composition and other product attributes
- **Forecasting models** that predict future demand, inventory needs, and operational trends

Decision-relevant information is often spread across:

- databases, dashboards, and planning systems
- documentation, audit logs, and operational reports
- model artifacts, training data, and deployment pipelines

Role of the Agentic Chatbot

An agentic chatbot can **democratize access** to these systems by allowing even non-technical users to inspect models, query data, run analyses, and understand recommendations through natural language.

What the Assistant Can Do

The assistant can answer questions such as:

- Which nodes are at highest stockout risk this week?
- Based on the model behavior and training data and audit, what are your **recommendations to improve the classifier**?
- If I change the optimization inputs, how will the **truck allocation** change?
- Can you run a **what-if analysis** with different capacity, demand, or cost assumptions?
- Can you trigger **retraining and redeployment** of the ML model under approval?
- How would forecast changes impact downstream transportation or inventory decisions?
- Can you summarize which assumptions are most responsible for the current planning output?

Key value

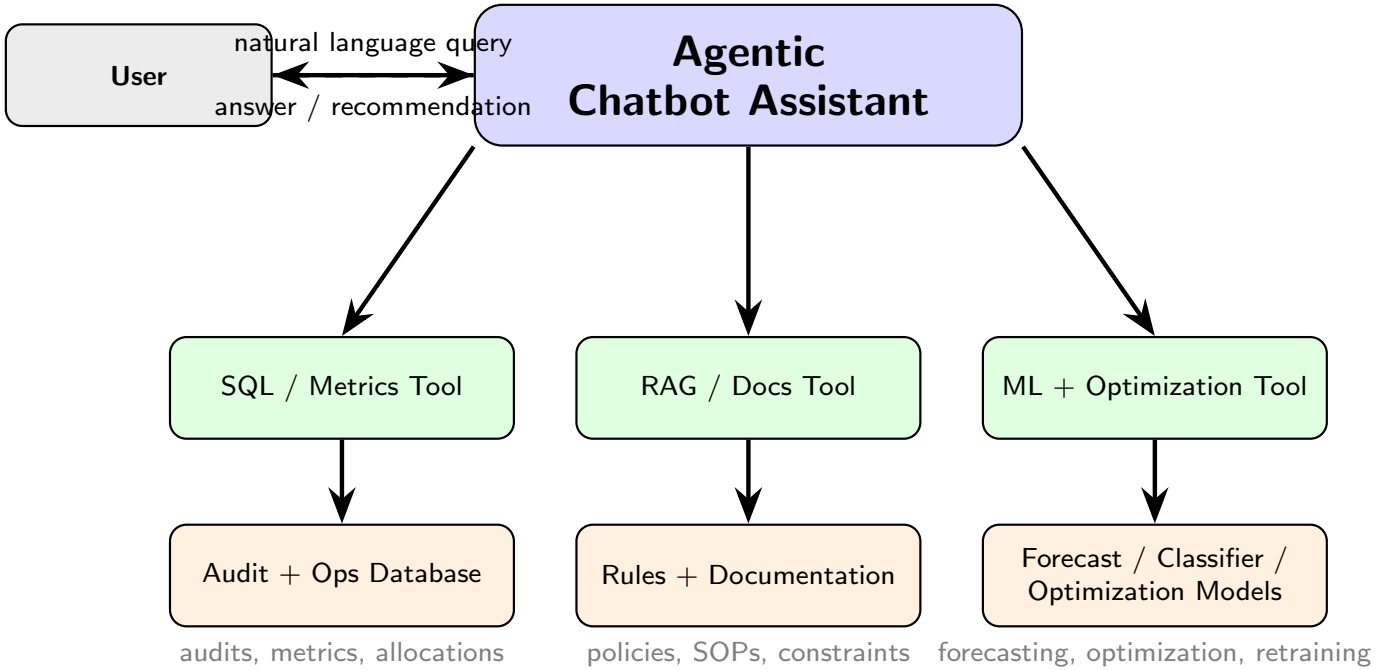
The value is not only faster and easier access to model outputs; it is the ability to discover why the model is wrong, where reality is richer than the formulation, and how to continuously close that **fidelity gap**.

Mapping the Example to Agentic AI Concepts

Concept	Supply chain example
Tools	Query audit databases, retrieve forecast data, call optimization solvers, run simulations, trigger ML pipelines
Skills	Misclassification analysis, what-if simulation, forecasting diagnostics, recommendation generation, mitigation planning
Memory	Prior defects, planner overrides, historical disruptions, past model versions, earlier recommendations
Guardrails	Mask sensitive data, block unsafe actions, require approval for re-training/redeployment, log actions for auditability
Human in the loop	Planner or operator approves high-impact actions such as model re-deployment, production changes, or operational overrides

The chatbot becomes agentic when it can reason across multiple sources, use tools, remember context, and act under guardrails.

Step 1: Agentic Chatbot



Example: "Which products were misclassified in the latest audit? What do you recommend to improve the model? If I reduce truck capacity by 10%, how will the allocation change?"

From Chatbot to Always-On Autonomous Agents

From user-driven interaction to persistent ownership

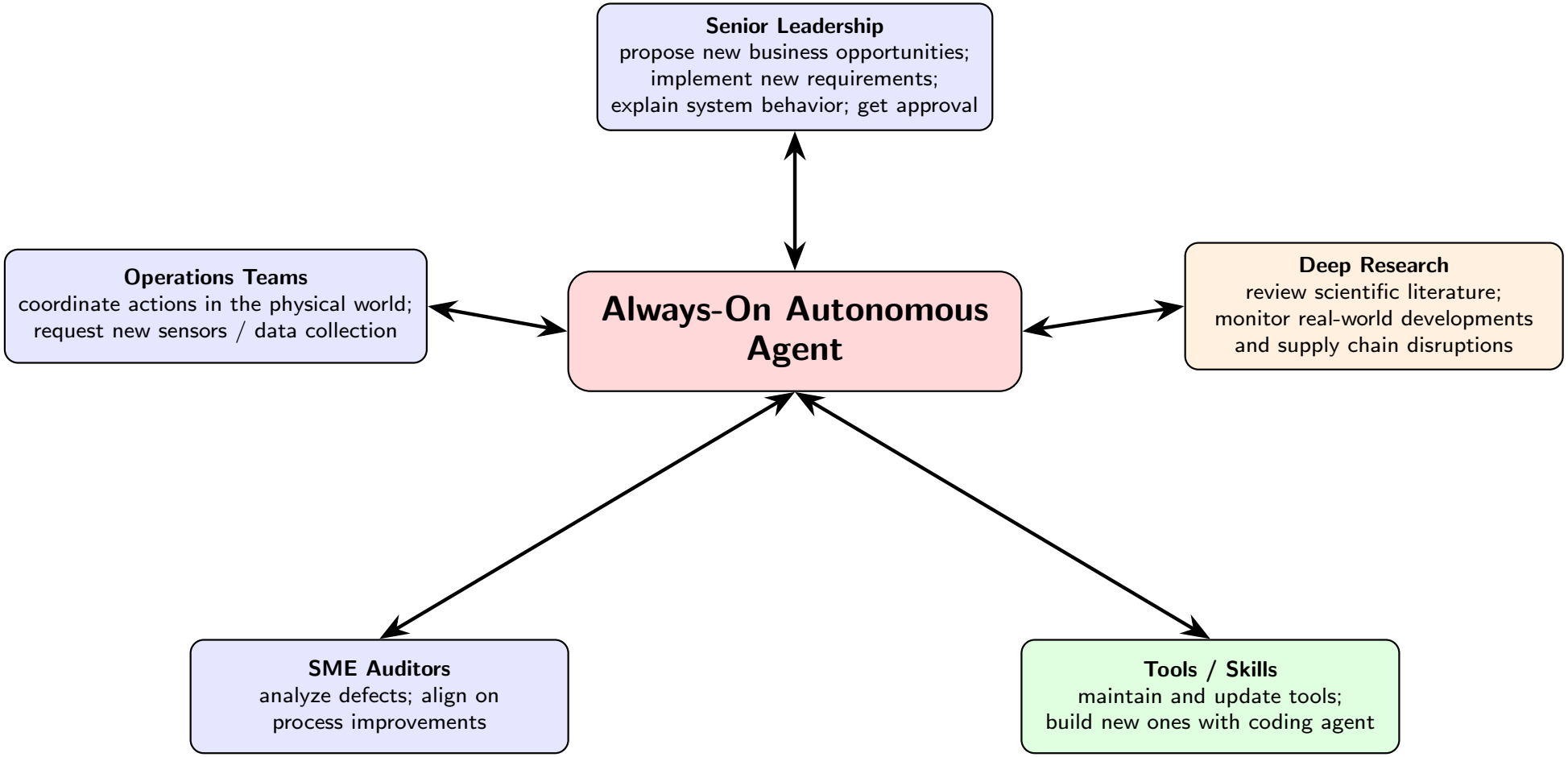
The chatbot assists a human when asked. An always-on, self-evolving, long-running autonomous agent continuously **monitors**, **improves**, and **maintains** the systems over time.

Example instruction to an autonomous agent:

> *"Hi Agent, you own the supply-chain processes from now on. Maintain and improve the current state of the systems, keep them up to date as the world evolves, review the scientific literature, run experiments to improve performance, interact with senior domain experts and operations teams in the physical world, incorporate new business initiatives into our models and systems, and proactively identify opportunities for expansion. You are equipped with tools and a coding agent: you may use existing tools, update them, or build new ones when needed. You may also coordinate with other agents or teams as part of the workflow. Make decisions under uncertainty within guardrails and permission policies"*

- **Always-on** rather than session-based
- **Self-improving** through experiments, monitoring, and feedback loops
- **Long-running** across changing business and operational conditions
- **Proactive** rather than only reactive

Step 2: Always-On Autonomous Agent



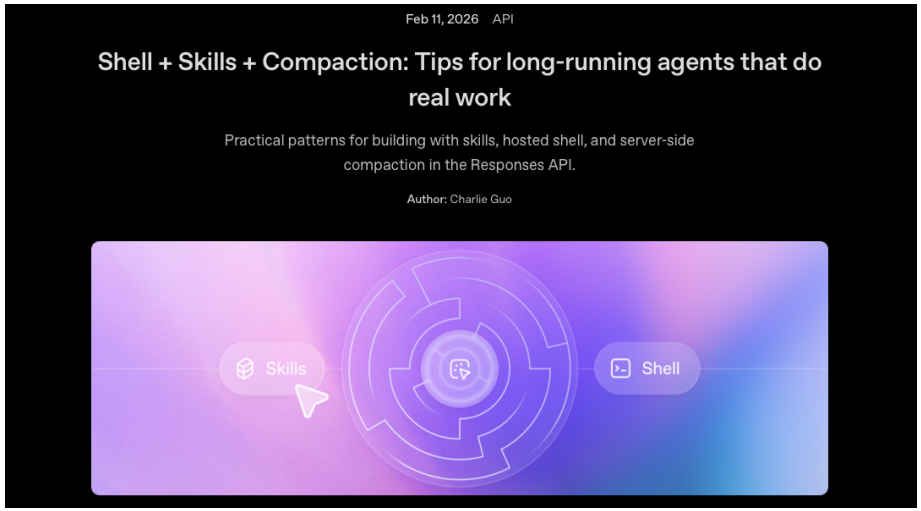
The agent continuously improves the system by interacting with leadership, physical-world operations, auditors, external knowledge, and its own tools.

Inspired by recent work on long-running, self-evolving agents, secure agent runtimes, skills, and shell-based execution.

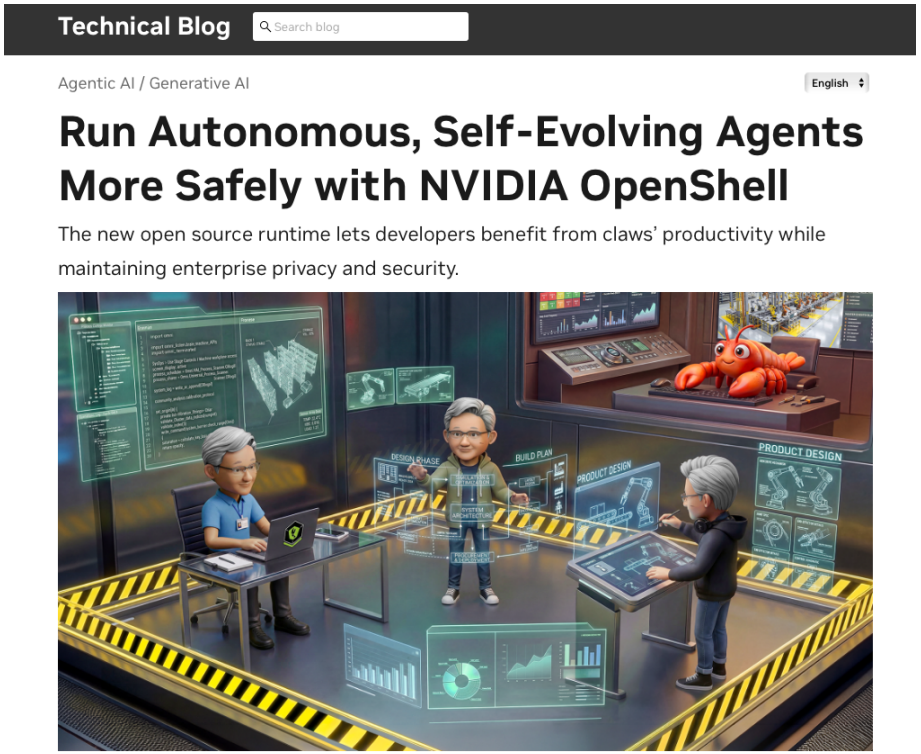
Always-On Autonomous Agents

Shift in Capability

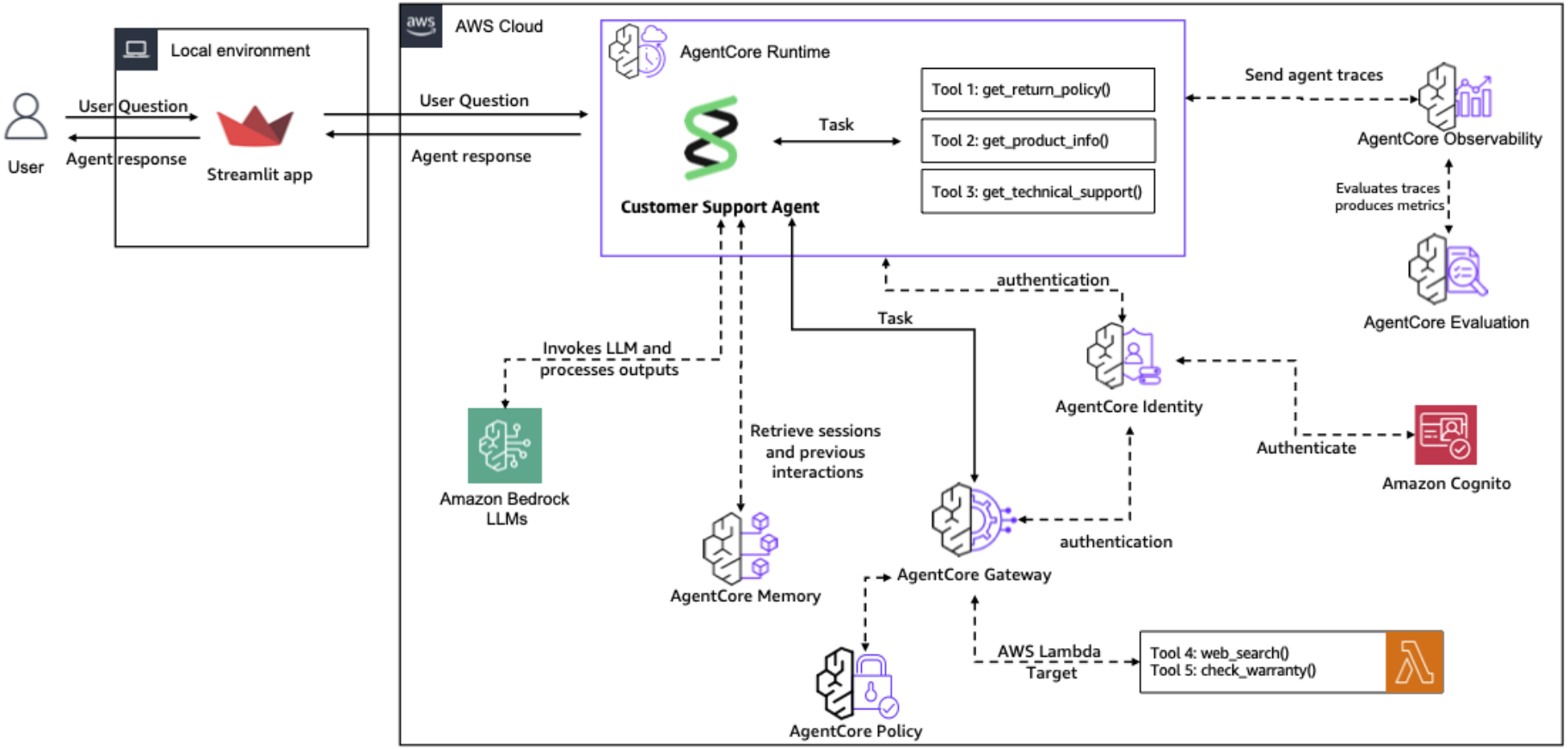
AI has evolved from assistants following your directions to agents that act independently. Called claws, these agents can take a goal, figure out how to achieve it, and execute indefinitely, while leaving you out of the loop.



Source: OpenAI developer blog, *Skills, shell tips, always-on autonomous agents*
`https://developers.openai.com/blog/skills-shell-tips`



Source: NVIDIA OpenShell
`https://developer.nvidia.com/blog/run-autonomous-self-evolving-agents-more-safely-with-n`

Lab: Building an Agentic AI Chatbot

From theory to practice: putting the agent loop, tools, and memory to work.